

Presentation #1:

Project Comprehension & Objectives

Akshay Anand, Ben Pilnick, Isabella Toth, Zoe Lively

Problem Statement

To confidentially identify the severity of potential eye cancer (pre-invasive melanoma) into 1 of 3 classifications (benign, low-grade, and high-grade) by looking at distribution, shape, and size of conjunctival melanocytic intraepithelial lesions (C-MIL) in a reproducible and accurate grading method.

Relevant Background

- Conjunctiva/type of cancer we are working with
 - C-MIL, pre-invasive melanoma in the eye (3 levels: benign, low-grade, and high-grade)
- Why is this project important
 - Improve accuracy in classifying C-MIL (benign, low-grade, high-grade) to decide correct treatment while minimizing unnecessary side-effects from incorrect diagnosis.
 - Only 39% of response were correct for classifying low-grade C-MIL (hardest to identify of 3 classifications) (WHO 2022)
 - Often subjectivity of pathologists, who may disagree in classifying the same C-MIL
- Multiple instance learning
 - ML robust to human errors (human subjectivity in diagnoses) as long as it captures all patterns that humans capture
 - MIL: treating patches of the same slice as correlated, slices of the same stain as correlated, and stains of the same case as correlated, i.e., there are multiple components of a single datapoint
 - Instances = patches, bags: slices $\rightarrow$ stains $\rightarrow$ case; labels = only at the case (bag) level
 - Study basis for MIL usage in C-MIL $\rightarrow$ strong support without many downsides

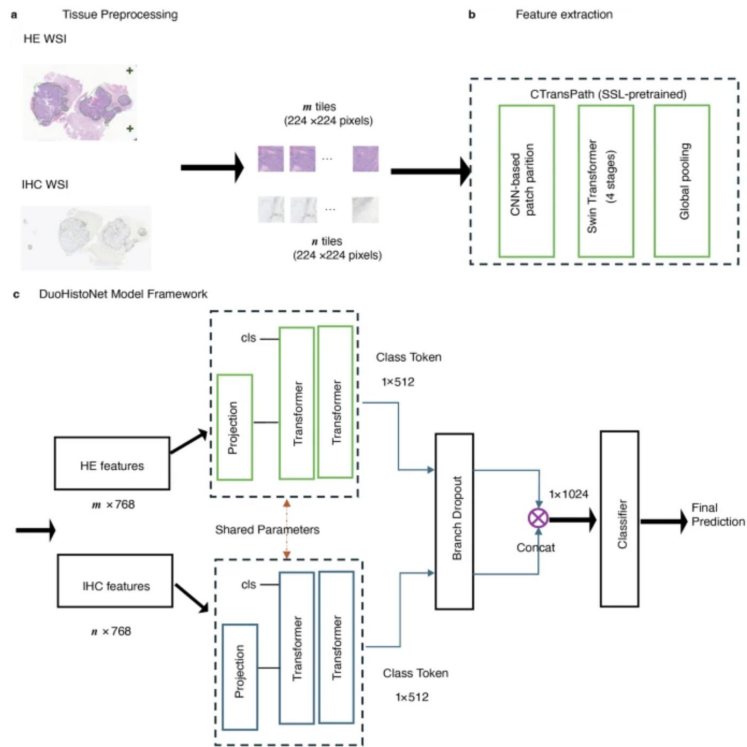
Relevant Background Con't

- **Immunohistochemistry** is a laboratory technique that uses antibodies to identify different types of cells in a tissue section
- 4 stains used on slices to identify melanocytes in C-MIL:
 - **H&E** → 1st stain pathologists see; shows relationship with other cells in addition to IHC features
 - **IHC with Melan-A** → shows architectural features; pathologists check if H&E comes back suspicious
 - **IHC with SOX10** → shows architectural and cytological features; pathologists check if H&E comes back suspicious
 - IHC with PRAME (we're not using this since data is unreliable)
 - **All 3 stains we're using are more or less equally important**

Study Basis

- Introduces **DuoHistoNet** → dual-modality transformer MIL model that analyzes both H&E and IHC whole-slide pathology images to predict key cancer biomarkers and patient outcomes in colorectal and triple-negative breast cancers
 - Uses **CTransPath** (CNN for local texture) as a tile feature extraction
 - Uses **Transformer MIL aggregation** (tile embeddings → transformer layers) to output a slide-level prediction using self-attention to model those relationships
 - **Processes H&E and IHC in parallel**; concatenates both feature streams for final prediction
 - **Branch dropout** (sometimes drops one modality during training → better generalization)

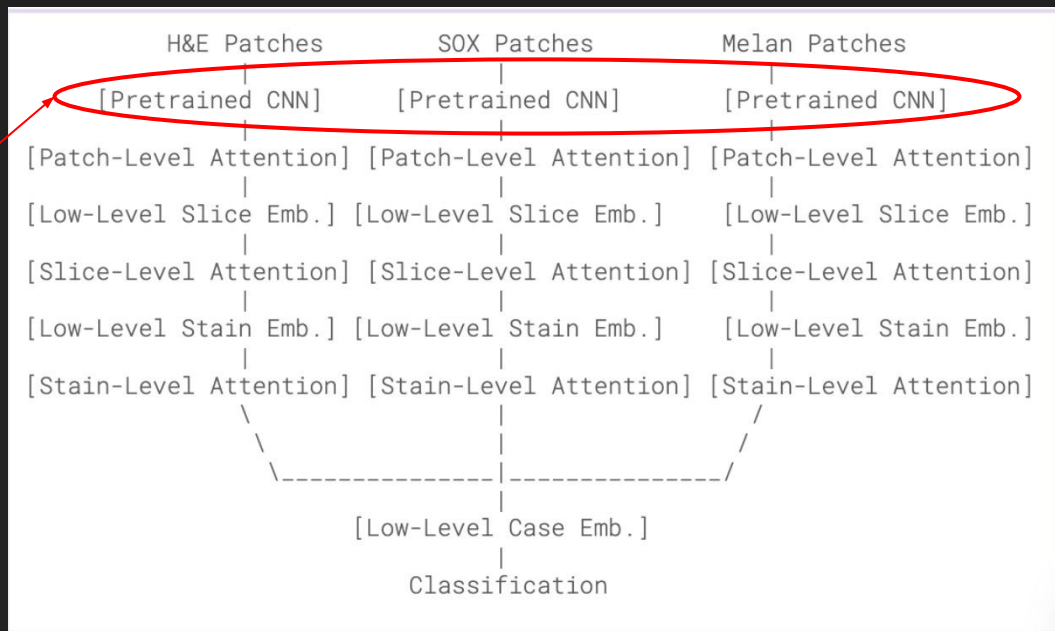
Fig. 1: Overview of the data pre-processing and model architecture.



Study Basis Con't

- **Shows better accuracy and prognostic power than using H&E or IHC alone**
 - And better pathologist scoring in some cases
- Very large data scale (~20,800 CRC, ~15,000 TNBC)
- CRC: dual-modality model had strongest survival stratification
- TNBC: stratified survival better than pathologist CPS scoring
- Dual-Modality helps when morphology and molecular status are tightly linked
 - **C-MIL can exploit this** because encourages model to learn:
 - Which patches are informative, as well as consistent or inconsistent across modalities
 - Doesn't require patch labels (instead of "which patch is positive" you get "which patches are consistent across H&E and IHC in a biomarker-relevant way?"
- Limitations:
 - Only MLH1 IHC used (not full 4-marker MMR panel)
 - No external dataset with paired H&E and IHC for validation
 - Retrospective, real-world data (no randomization)
- Takeaways:
 - Transformer-based MIL benefits from better patch embeddings
 - Dual-modality improves biomarker predictions at the patch level
 - **Learning cross-modal patch representations via C-MIL should further improve slide-level prediction.**

Multi Stain Model Overview



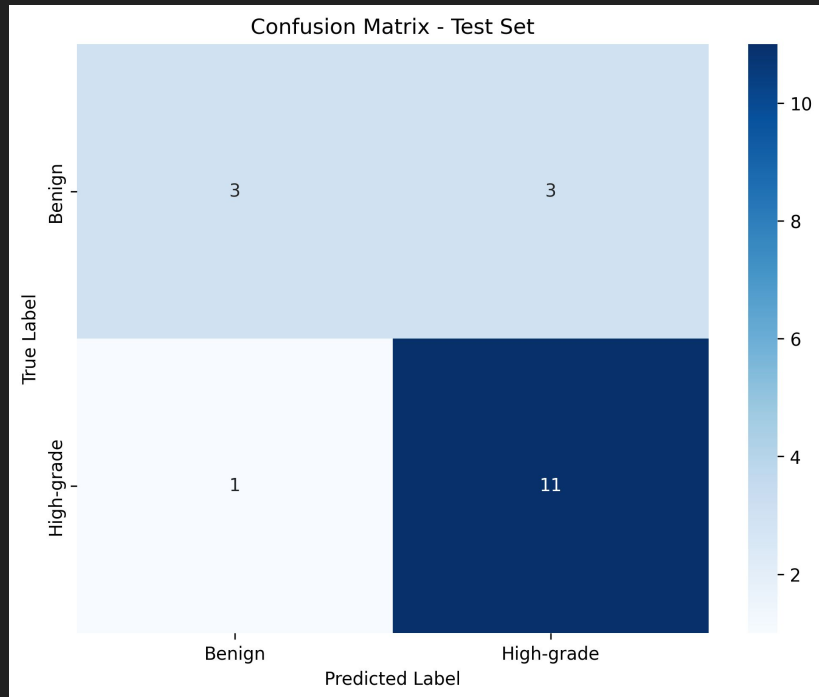
DenseNet121

Patches -> Slices -> Stain -> Case -> Classification (Benign/High Grade)

Code Execution

Initialization/Setup 	main.py , utilz.py	- Parses arguments default in config.py. Can override arguments when running main.py. - Set up device and creates the output directories
Data Preparation 	dataset.py , dataset_utilz.py	- Loads the patch/labels files. Builds directory. - Splits the data into train, test, and validation - Creates a pytorch dataset. Transforms file paths to tensors to be inputted into the model.
Model preparation 	model.py	- Pre Trained DenseNet 121 on patches. - Attention pooling patches -> slices -> stains -> cases - Final classifier (softmax)
Training infrastructure 	trainer.py	- Adam Optimizer (weighted cross-entropy loss, learning rate scheduler - Early stopping mechanism
Training 	trainer.py	- For each Epoch, train all the training cases one by one. Calculate the loss using cross-entropy, backpropagate gradients. Update the weights, and validate on the val. Set. Adjusts the learning rate if needed, and uses early stopping if loss is not improved.
Evaluation	trainer.py , attention_analysis.py	- Test set trained on the model. Obtain the test loss and accuracy. - Attention analysis addons.

Benign Cases are frequently misclassified as high-grade



Test Loss	Test Accuracy	# of Samples
0.73	0.78	18

High-Grade Recall	Benign Recall
$11/12 = 0.917$	$3/6 = 0.50$

Potential Changes to the Code

Eliminate resizing the images to 224x224, let height/weight be variable

- Filter out the images that have less than 32x32.
- A custom `collate_fn` already exists to tell the Data Loader how to stack the embeddings.
- This preserves the true structure of the data without padding or masking, which results in more interpretable attention weights.

Potential Changes to the Code

Redundant for loops in [main.py](#)

- Slice to class iterates three separate times. This computation time can be quite slow.
- Instead of three separate loops, have one loop that loops once and creates a tuple of the case id and the slice id.

```
test_cases_set = set(test_cases)

# Map loaded case IDs back to slices
train_slices = [(case_id, slice_id) for (case_id, slice_id) in slice_to_class.keys() if case_id in train_cases_set]
val_slices = [(case_id, slice_id) for (case_id, slice_id) in slice_to_class.keys() if case_id in val_cases_set]
test_slices = [(case_id, slice_id) for (case_id, slice_id) in slice_to_class.keys() if case_id in test_cases_set]
```

```
train_slices, val_slices, test_slices = [], [], []

for (case_id, slice_id) in slice_to_class.keys():
    if case_id in train_cases_set:
        train_slices.append((case_id, slice_id))
    elif case_id in val_cases_set:
        val_slices.append((case_id, slice_id))
    elif case_id in test_cases_set:
        test_slices.append((case_id, slice_id))
```

Identifying Areas of Improvement

GENERALIZED:

- Accuracy rates and prediction performance not high enough
 - ** ultimate goal! **
- Redundant/inefficient code
- Plateaued progress, baseline fine-tuning required

TARGETED:

- Unused data from combos when stains have >5 slices
- L2 regularization $\rightarrow$ weight_decay:
 - Manual configs, acts on model params not attention outputs
 - Dropout introduces noise for attention mechanism, does not directly affect attention distribution

Next Steps

- Use More of Available Data
 - Split cases into train-only sub-cases to incorporate unused slice data
 - Target 28 case/stain combinations with >5 slices
 - Implement in build_case_dict() ([data_utils.py](#))
 - Give special attention to benign cases, heavy unused data
- Improve Model Interpretability
 - Run patch-by-patch predictions to understand individual patch contributions
 - 100% attention to single patches or run without attention layers
 - Use results to validate attention behavior and support model decisions

Next Steps (cont.)

- Strengthen Regularization
 - Current approach: L2 weight decay + dropout
 - Explore attention-specific regularization:
 - Elastic net penalties on attention logits
 - Entropy regularization to prevent overly sparse attention
 - Tune weight_decay and learning_rate
 - Implement in AttentionPool ([models.py](#))
- Optimize Hyperparameter Selection
 - Use Optuna for hyperparameter tuning
 - Wrap main() ([main.py](#)) as Optuna objective
 - Compare Optuna-tuned L2 vs. alternative regularization strategies

Next Steps (cont.)

- Long-Term Extension
 - Experiment with alternative attention mechanisms
 - gated, multi-head, transformer-style
 - Evaluate custom loss functions in MILTrainer ([trainer.py](#))

Mathematical Background

Main idea:

How does the MIL model utilize attention weights, data augmentation, and hyperparameters to selectively focus on the areas that matters?

- Why do we need MIL and Hierarchical attention?
- What is the mathematical process in which certain weights are classified and lead to a prediction?
- How does the model work doing training vs testing?
- What does data augmentation provide for us?
- How do changes in hyperparameters affect our model?

Basics of the model

How the data is stored in the model

- Case ID
 - Stains: Melan, Sox 10, H&E
 - Slices
 - Patches

How the model works:

[1] Images for each patch placed in CNN after augmentation

[2] The CNN outputs and embedding vector for each patch

[3] The patch weights within each slice create a embedding vector for each slice

[4] This process repeats up the ladder to the stain and then the case

[5] Logit values are created in order to avoid vanishing gradient problem and provide values to determine which class this case should be listed as

Data Augmentation

Possible data augmenting tactics we can use:

```
transforms.RandomResizedCrop (IMAGE_CONFIG['image_size'][0]),  
transforms.RandomHorizontalFlip(),  
transforms.RandomVerticalFlip(),  
transforms.RandomRotation(15),  
transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2)
```

Keys of data Augmentation:

[1] First step before the use of CNN

[2] Each patch is randomly transformed given one of these changes

[3] The randomness and constant variety of the same image forces the CNN to learn many patterns as opposed to memorize the same concept

Training vs Testing the model

Extremely important distinction between these two stages:

During training we are constantly experimenting:

- Data augmentation
- Batch Normalization
- Dropout
- Learning via gradient and adjusting based on Logit values
 - Smaller difference between values leads to higher gradient changes and bigger difference between values leads to smaller gradient changes

and

During testing we are testing the model on what it has learned:

- No augmentation
- No Batch normalization - “First, the noise in the sample mean and the sample variance arising from estimating each on mini batches is no longer desirable once we have trained the model” - Dive into Deep Learning textbook. [Link in citation page](#)
- No Dropout - “Typically, we disable dropout at test time. Given a trained model and a new example, we do not drop out any nodes and thus do not need to normalize.” - Dive into Deep Learning Textbook.
- We calculate the most important patches not based on just patch importance from the embedding, but rather form this equation:

$$\text{effective_value} = \text{stain_attn_value} * \text{slice_attn_value} * \text{patch_attn_value}$$

The Role of Hyperparameters

- Not about what the model learns, but rather how the model learns:
- There are a myriad of important hyperparameters, but the 5 that have a lot of influence over the model are
 - Embed_dim: the amount of information that a patch can take forward – can easily cause overfitting(too large) and underfitting (too small)
 - Attention_hidden_dim: controls the attention scoring function and its ability to fit properly – too small can be high bias and too large can be high variance
 - Per_slice_cap: The largest number of patches that we allow to be calculated for a slice. If the number of patches becomes too large, the importance of a high grade patch can become diluted to the point where we may miss certain certain cases of cancer
 - Dropout: randomly removes features through training to force the model to learn.
 - Learning_rate + scheduler: This combination determines how much the model can explore and then stabilize once it finds a spot that is quite accurate

References

Final_presentation_Fall2025.pdf

CMIL_classification_Problem_statement_and_Data_Winter2026.pdf

Cheng, Yating, et al. “Synergistic H&E and IHC image analysis by AI predicts cancer biomarkers and survival outcomes in colorectal and breast cancer.” Communications Medicine, vol. 5, no. 1, 24 Oct. 2025, <https://doi.org/10.1038/s43856-025-01045-9>.